

📁 Meeting Management System - Learning Journal (Complete Phase 1)

Welcome to your vibe-coding Learning Journal! This document is a comprehensive, stage-by-stage guide of everything we built from the very first day of the project up until the end of Phase 1.

If you are a beginner, use this as a study guide to understand **what** features we built, **why** we built them, and **how** the underlying code actually works.

🏗️ Stage 1: The Foundation & Authentication

What we built:

We set up a modern full-stack application using **Next.js** (Frontend) and **Node.js/Express** (Backend), connected to a **MongoDB** database. We then built a secure Login system.

How it works (The Code):

- **The Tech Stack (MERN+Next):** We used React (via Next.js) for the user interface because it makes building interactive components easy. We used Express.js to create API endpoints (`/api/auth/login`) that our frontend talks to. MongoDB is our NoSQL database where we store JSON-like documents.
- **Authentication (JWT):** When a user logs in with their email and password, the backend verifies the password using `bcrypt` (which hashes passwords for security). If correct, the server generates a **JSON Web Token (JWT)**. Think of a JWT as a digital wristband. The user keeps this wristband in their browser's `localStorage` and shows it to the server every time they try to access a private page.
- **Role-Based Access Control (RBAC):** Inside the JWT and the database, every user has a `Role` (SuperAdmin, DepartmentAdmin, Employee). This role determines what buttons they can click.

📊 Stage 2: Role-Based Dashboards

What we built:

After logging in, users are taken to a central dashboard. However, a SuperAdmin sees global statistics and user management tables, while a regular Employee only sees the meetings they are personally invited to.

How it works (The Code):

- **Conditional Rendering:** In React, we use simple `if` statements and ternary operators (`condition ? true : false`). For example: `{currentUser.role === 'SuperAdmin' && <AdminWidget />}`. This ensures the browser physically does not render the admin buttons for regular employees.
- **Dashboard Analytics:** For the Admin charts, we wrote backend aggregation pipelines. MongoDB counted how many meetings were "Completed" vs "Scheduled" and returned those raw numbers to the frontend to display in the colorful statistics cards.

🗄️ Stage 3: The Core Meeting Engine

What we built:

The ability to Create, Read, Update, and Delete (CRUD) meetings. Organizers can fill out a form with dates, times, venues, and select participants to invite.

How it works (The Code):

- **Mongoose Schemas:** We defined a `MeetingSchema` in the backend. This strict blueprint forces every meeting to have a `title`, `date`, `organizerId`, and an array of `participants`.
- **RSVP System:** Inside the `participants` array, each user has a `status` field that defaults to `Pending`. When an employee clicks "Accept" on their dashboard, the frontend sends a `PUT /api/meetings/:id/rsvp` request. The backend finds that specific user in the array and changes their status to `Accepted`.
- **API Routing:** `Next.js app/meetings/[id]/page.tsx` uses dynamic routing. The `[id]` part means the URL can be `/meetings/12345`. Next.js grabs 12345 from the URL and fetches that specific meeting from the backend.

📅 Stage 4: Agenda Management

What we built:

A meeting isn't just a calendar event; it needs an Agenda. Participants can propose agenda items (topics to discuss), and the Organizer can "Approve" or "Reject" them.

How it works (The Code):

- **Relational Data:** Instead of stuffing the agendas directly inside the Meeting document, we created a separate `AgendaSchema`. Every agenda item has a `meetingId` pointing back to the parent meeting. This keeps the database fast and organized.
- **Status Workflow:** When an employee submits an agenda, it saves as `Pending`. The Organizer sees a "Review Agendas" table. Clicking "Approve" updates the database to `Approved`, which officially adds it to the Meeting's timeline and allocates the requested time (e.g., 15 minutes).

🚀 Stage 5: "Enterprise" Professional Upgrades

To make the app feel like a premium, paid product, we added several advanced features:

A. Rich Text Editors (`react-quill-new`)

- **The Feature:** We replaced boring plain-text boxes with Word-style formatting editors.
- **How it Works:** The editor saves your text as raw HTML (like `<strong>Hello</strong>`). To display this HTML safely in Next.js, we use a special React property called `dangerouslySetInnerHTML`. Because we use SSR (Server-Side Rendering), we had to use `Next.js dynamic()` imports to tell the server *not* to load the editor until it reached the user's browser.

B. Recurring Meetings Engine

- **The Feature:** Organizers can select "Weekly" and generate 10 future meetings instantly.
- **How it Works:** Instead of saving to the database one by one (which is very slow), our backend uses a `for` loop and JavaScript `Date` math to calculate the future dates. It puts all those new meetings into an array and uses MongoDB's `insertMany()` to blast them all into the database in one lightning-fast operation.

C. Drag & Drop Agendas (`@dnd-kit`)

- **The Feature:** Organizers can click and drag agenda items up and down to reorder the meeting flow.
- **How it Works:** We wrapped our React list in a `<DndContext>`. When an item is dropped, we use an `arrayMove` function to instantly swap the items on the screen. Then, we secretly fire an API request in the background (`bulkWrite`) to update the `sequence` numbers (1, 2, 3...) of all the agendas in the database simultaneously.

D. Calendar Emails & Exports (`.ics`, PDF, Excel)

- **The Feature:** Automated calendar invites and downloadable meeting packets.
- **How it Works:** We integrated an `ics` Node library to compile the meeting dates into a standard `text/calendar` file. We attach this to a `nodemailer` email payload. For the frontend PDF exports, we used `jspdf-autotable`. We wrote a custom `stripHtml` regex function (`/<[^\>]+>/g`) to instantly delete the invisible rich-text HTML tags so the PDF wouldn't crash when trying to print the text!

🔒 Bonus Lesson: Git Secret Scanning

During development, we accidentally ran `git add .` while our `server/.env` file was sitting out in the open. The `.env` file holds highly sensitive passwords (like our MongoDB connection URL and Email Passwords).

GitHub has a built-in AI called **Secret Scanning**. It scanned our commit as we were pushing it, detected the passwords, and **blocked our push** to protect us from getting hacked!

How we fixed it:

1. We used `git reset HEAD~1` to undo our broken commit.
2. We used `git rm --cached .env` to surgically remove the file from Git's memory.
3. We fixed a file-encoding glitch in our `.gitignore` file, added `.env` to it, and pushed safely!

Always double-check your `git status` before committing to ensure `.env` isn't listed!